CANVIA

CAPACITACIÓN QA AUTOMATION · PARTE 2 · SESIÓN 4 DE 4

Reportes, pipeline y examen final

Cada push y cada Pull Request corren las pruebas en GitHub Actions, dejan el reporte como artefacto, resumen el resultado en el PR y publican el HTML en GitHub Pages. Después: el examen.

Instructor **Giancarlo Palomino**

Viernes 30 de octubre · 17:30 a 18:45

GitHub Actions · Playwright 1.64 · Cucumber 13

75 MINUTOS

Del verde en tu PC al verde en la CI

25 min — **GitHub Actions**
Caché de npm, Chromium en la CI, reporte y capturas como artefactos, resumen en el PR y publicación en GitHub Pages.

45 min — **Examen final**
Sobre la web del Portal Canvia Market, en tu rama `feature/examen-<tu-nombre>`.

5 min — **Cierre**
Qué sigue después del curso.

REQUISITO

Tu rama con los ejercicios 4 y 5 en verde (`npm test`). GitHub Pages lo activa el instructor una vez en el repo del curso.

AL TERMINAR SABRÁS

Leer un pipeline línea por línea y dejar evidencia pública de cada ejecución.

QUÉ CORRE Y CUÁNDO

Rápido en el PR, completo en la rama

EVENTO	QUÉ CORRE	PAGES
`pull_request`	**Solo** `@smoke` (2 escenarios, segundos): feedback rápido para la revisión	No
`push` a `main` o `develop`	Regresión completa (`npm test`, 20 escenarios)	Sí
`schedule`	Regresión completa, lunes a viernes a las 06:00 de Lima: detecta cambios de la app aunque nadie suba código	Sí
`workflow_dispatch`	Manual desde la pestaña Actions, con un campo opcional de etiquetas	Sí

.github/workflows/ci.yml · on — YAML

```yaml
on:
  push:
    branches: [main, develop]
  pull_request:
  schedule:
    - cron: '0 11 * * 1-5' # lunes a viernes, 06:00 de Lima (11:00 UTC)
  workflow_dispatch:
    inputs:
      tags:
        description: 'Expresión de etiquetas (vacío = regresión completa)'
        required: false
        default: ''
```

EL CRON VA EN UTC

Lima es UTC–5: las 06:00 de Lima son las 11:00 UTC.

DOS JOBS, ONCE PASOS

El pipeline paso a paso

JOB PRUEBAS

1
checkout
trae el código

→

2
setup-node
Node 24 + caché npm

→

3
npm ci
lo exacto del lock

→

4
Chromium
con dependencias

→

5
typecheck
tipos antes de correr

→

6
escenarios
@smoke o todo

7 · SIEMPRE
resumen
JUnit → tabla en el PR

→

8 · SIEMPRE
artefacto
reports/ con capturas y trazas

→

9 · FUERA DEL PR
sitio
index.html

→

10 · FUERA DEL PR
upload-pages-artifact
empaqueta el sitio

→

11 · JOB PUBLICAR
deploy-pages
URL pública del reporte

IF: ALWAYS()
Los pasos 7 a 10 corren aunque fallen las pruebas: el reporte rojo es el que más se necesita.

TYPECHECK PRIMERO
Un error de tipos falla en segundos, sin gastar minutos de navegador.

DOS JOBS
`publicar` necesita permisos de Pages que `pruebas` no debe tener.

PREPARAR LA MÁQUINA

Caché de npm y Chromium en la CI

.github/workflows/ci.yml · job pruebas — YAML

```yaml
jobs:
  pruebas:
    name: typecheck + cucumber
    runs-on: ubuntu-latest
    timeout-minutes: 20
    steps:
      - uses: actions/checkout@v7

      - uses: actions/setup-node@v7
        with:
          node-version: 24
          cache: npm

      - name: Instalar dependencias
        run: npm ci

      - name: Instalar Chromium
        run: npx playwright install --with-deps chromium

      - name: Revisar tipos
        run: npm run typecheck
```

CACHE: NPM

Guarda la carpeta de descargas de npm usando `package-lock.json` como llave. La segunda ejecución instala en segundos.

NPM CI, NO NPM INSTALL

Instala exactamente lo del lock y falla si el lock no coincide con `package.json`. Lo que corre en la CI es lo que probaste.

--WITH-DEPS CHROMIUM

Solo Chromium (no los tres navegadores) y las librerías de Linux que necesita para arrancar.

VERSIONES

Las *actions* se fijan por versión mayor (`@v7`): reciben correcciones sin cambios que rompan.

CORRER Y DEJAR EVIDENCIA

Etiquetas según el evento, evidencia siempre

.github/workflows/ci.yml · extracto — YAML

```yaml
      # Pull Request: solo @smoke. Push, schedule y manual: regresión completa.
      - name: Ejecutar escenarios
        env:
          TAGS: ${{ github.event_name == 'pull_request' && '@smoke' || inputs.tags }}
        run: |
          if [ -n "$TAGS" ]; then
            echo "Etiquetas: $TAGS"
            npm test -- --tags "$TAGS"
          else
            echo "Regresión completa"
            npm test
          fi

      - name: Resumen en el job y en el Pull Request
        if: always()
        run: node scripts/resumen-junit.mjs reports/cucumber-report.xml

      - name: Subir reporte, capturas y trazas
        if: always()
        uses: actions/upload-artifact@v7
        with:
          name: reporte-cucumber
          path: reports/
          if-no-files-found: ignore
          retention-days: 14
```

TAGS

En un PR vale `@smoke`; en un push vale lo escrito a mano (vacío = todo). `cucumber.js` suma siempre `not @bug`.

SCRIPTS/RESUMEN-JUNIT.MJS

Lee el JUnit de Cucumber y escribe una tabla Markdown en `$GITHUB_STEP_SUMMARY`. Sin dependencias: Node puro.

ARTEFACTO

Un zip descargable desde la ejecución con HTML, JSON, JUnit, capturas y trazas. Dura 14 días.

PRUÉBALO EN TU PC

`node scripts/resumen-junit.mjs` imprime en la consola la misma tabla que verás en GitHub.

LO QUE VE QUIEN REVISA

El resultado, sin abrir los logs

```
$ npm test && node scripts/resumen-junit.mjs
### Pruebas en verde

| Escenarios | Pasaron | Fallaron | Omitidos | Duración |
|---|---|---|---|---|
| 20 | 20 | 0 | 0 | 16.0 s |
```

```
$ npm run test:bugs; node scripts/resumen-junit.mjs
### Pruebas con fallas

| 5 | 0 | 5 | 0 | 8.0 s |

| Característica | Escenario | Motivo |
| Caza de bugs con usuarios problemáticos | El catálogo se ordena por za (problem_user) | El catálogo no está ordenado por "za" |
```

DÓNDE SE VE

En la ejecución (pestaña *Summary*) y desde el PR: *Checks* → *CI* → *Summary*.

QUÉ MUESTRA

Totales, duración y, por cada escenario que falló, la característica, el escenario y el motivo de la aserción.

BRANCH PROTECTION

Con la regla "la CI debe pasar" en `develop`, un PR en rojo no se puede fusionar.

ALTERNATIVA

Actions conocidas como `dorny/test-reporter` leen el mismo JUnit; aquí usamos un script propio para que se entienda cada línea.

UN ENLACE QUE CUALQUIERA PUEDE ABRIR

Publicar el reporte en GitHub Pages

.github/workflows/ci.yml · extracto — YAML

```yaml
      - name: Preparar el sitio del reporte
        if: always() && github.event_name != 'pull_request'
        run: |
          mkdir -p sitio
          cp reports/cucumber-report.html sitio/index.html

      - name: Subir el sitio para GitHub Pages
        if: always() && github.event_name != 'pull_request'
        uses: actions/upload-pages-artifact@v5
        with:
          path: sitio/

  publicar:
    name: publicar reporte en GitHub Pages
    needs: pruebas
    if: always() && github.event_name != 'pull_request' && needs.pruebas.result != 'cancelled'
    runs-on: ubuntu-latest
    permissions:
      pages: write
      id-token: write
    environment:
      name: github-pages
      url: ${{ steps.despliegue.outputs.page_url }}
    steps:
      - id: despliegue
        uses: actions/deploy-pages@v5
```

POR QUÉ NO EN EL PR

Pages tiene un solo sitio por repo: lo publica la rama, no cada propuesta.

PERMISOS MÍNIMOS

El workflow tiene `contents: read`; solo `publicar` recibe `pages: write` e `id-token: write`.

RESULTADO

`https://gpalominoh-canvia.github.io/canvia-playwright-bdd/` con el último reporte de `develop`, también si falló.

UNA VEZ POR REPO

Settings → Pages → Source: GitHub Actions. Y en *Environments → github-pages*, permitir la rama `develop`.

25 MINUTOS

Guion de la demo

1	0–3	Activar Pages (*Source: GitHub Actions*) y permitir `develop` en el environment `github-pages`.
2	3–9	Recorrer `ci.yml` de arriba abajo con la lámina del pipeline al lado.
3	9–13	Rama `feature/demo-ci`, cambiar un texto del README, push y abrir el PR: corre solo `@smoke`.
4	13–17	Mostrar el resumen en *Checks* y descargar el artefacto `reporte-cucumber`.
5	17–21	Romper un `@smoke` a propósito, push: rojo, resumen con el motivo y la traza en el artefacto.
6	21–25	Fusionar a `develop` (ya en verde): regresión completa y enlace de Pages en el job `publicar`.

```
$ git switch -c feature/demo-ci
$ git commit -am "ci: prueba el pipeline"
$ git push -u origin feature/demo-ci
$ gh pr create --base develop --fill
$ gh run watch
```

PLAN B

Si la red falla, mostrar una ejecución anterior ya terminada (verde y roja) desde la pestaña Actions.

LEER EL ROJO DE LA CI

Seis fallas de pipeline y su causa

`Executable doesn't exist at …/chromium`

Falta el navegador

No está el paso `npx playwright install --with-deps chromium`.

`npm ci · lock file out of sync`

Lock desactualizado

Agregaste una librería sin subir `package-lock.json`. Corre `npm install` y súbelo.

`error TS2307: Cannot find module`

Mayúsculas en imports

Windows perdona `./tareas/iniciarSesion`; Linux no. El nombre tiene que coincidir exacto.

`Get Pages site failed · Not Found`

Pages sin activar

Settings → Pages → Source: GitHub Actions.

`Branch "develop" is not allowed to deploy`

Environment protegido

Agrega `develop` en *Environments → github-pages*.

`Pasa en mi PC, falla en la CI`

Dependencia del orden o del tiempo

En la CI todo es más lento y en limpio. Busca esperas fijas y escenarios que dependan de otros.

45'

Examen final

Portal Canvia Market: una tienda nueva, con bugs sembrados. Todo lo de la semana, en una sola entrega.

CÓMO FUNCIONA

45 minutos, una rama, un Pull Request

MOMENTO	QUÉ HACER
Minuto 0	Abrir el enunciado en `/examen` del portal. Crear `feature/examen-<tu-nombre>` desde `develop`.
0 a 35	Escenarios en Gherkin, Screenplay y `npm test` en verde contra el portal.
35 a 40	Documentar los bugs encontrados.
40 a 45	Push y Pull Request a `develop`. Lo que no esté subido al minuto 45 no cuenta.

PERMITIDO

Tu código de la semana, la documentación de Playwright y Cucumber, `/docs` del portal y las láminas.

NO PERMITIDO

Copiar código de un compañero o trabajar en pareja.

PORTAL CANVIA MARKET

Web en español con selectores `data-test`. Corre con `node server.js` en el puerto `3000` o con `docker compose up`; el instructor también la levanta para toda la sala.

apuntar las pruebas al portal — POWERSHELL

```
$env:BASE_URL="http://localhost:3000"; npm test
```

USUARIO	COMPORTAMIENTO
`ana`	Normal
`bloqueado`	No puede entrar
`lento`	Respuestas de 1,5 s
`errores`	Interfaz con bugs

Clave de todos: `canvia123`. El enunciado completo se publica en el portal al iniciar.

APROBACIÓN: 70 %

Cómo se califica

CRITERIO	PESO	QUÉ SE MIRA
Gherkin	20 %	Escenarios claros, en español y declarativos: sin selectores ni clics.
Screenplay	30 %	Tareas, interacciones y preguntas separadas; steps que solo delegan en el actor.
Flujos	20 %	Funcionan login, búsqueda, carrito y compra.
Pipeline	20 %	CI en verde con el reporte publicado.
Bugs	10 %	Encontrados y documentados (título, pasos, esperado, obtenido, severidad).

70 %

para aprobar el curso

PIPELINE DEL EXAMEN

Tu CI levanta el portal con el paso `uses: gpalominoh-canvia/canvia-portal-examen@main` antes de `npm test`. "Publicado" en tu PR = resumen en *Checks* + artefacto `reporte-cucumber`.

CONSEJO

Primero un escenario de login en verde y la CI publicando. Después crece. Un flujo completo vale más que cinco a medias.

LOS PRIMEROS 5 MINUTOS

Checklist para arrancar sin perder tiempo

- [ ] `git switch develop`, `git pull` y `git switch -c feature/examen-<tu-nombre>`.
- [ ] El portal responde: `http://localhost:3000/api/salud` → `{ "estado": "ok" }`.
- [ ] `BASE_URL` apunta al portal (los hooks ya la leen).
- [ ] Nuevos Targets en `ui/` con los `data-test` del portal: `usuario`, `clave`, `ingresar`, `error`…
- [ ] Un `.feature` nuevo, en español, con su etiqueta.
- [ ] Reutiliza el núcleo: `Actor`, `NavegarLaWeb`, `Clic`, `Escribir`, `TextoDe`.
- [ ] `npm run typecheck` y `npm test` en verde antes de cada push.

PANTALLAS DEL PORTAL

`/` login · `/catalogo` · `/carrito` · `/checkout` · `/pedidos` · `/docs` · `/examen`.

SAUCEDEMO NO SE TOCA

Tus escenarios de la semana siguen en el repo. Para el examen, separa por etiqueta y por carpeta de `ui/`.

LOS BUGS CUENTAN

Si algo se comporta raro con el usuario `errores`, escríbelo como escenario `@bug` y en tu `BUGS.md`.

CANVIA

CIERRE DEL CURSO

Del escenario en español al reporte publicado

Playwright
Selectores con significado y esperas que no adivinan.

Gherkin
Escenarios que negocio entiende y que fallan cuando deben.

Screenplay
Actores, tareas y preguntas: código que escala.

CI
Cada cambio probado, con evidencia pública.

Documentación: **playwright.dev** · **cucumber.io/docs** · **serenity-js.org**

Repositorio: **gpalominoh-canvia/canvia-playwright-bdd**